

DOI-USGS / dataretrieval-python

Q

Type ↵ to search

<>

Issues 18

Pull requests 16

Actions

Projects

Security and quality

Insights

 **dataretrieval-python** Public

Watch 11

Fork 63

Star 267

About

Python package for retrieving water data from USGS or the multi-agency Water Quality Portal

doi-usgs.github.io/dataretrieval-python/

Readme

License

Contributing

Activity

Custom properties

267 stars

11 watching

63 forks

Report repository

Releases 24

v1.3.0

Latest

last month

+ 23 releases

Deployments 224

github-pages

2 weeks ago

Packages

No packages published

Used by 140



+ 139

Contributors 31



+ 17 contributors

Languages

Python 100%

m...

7 Branches

25 Tags

Go to file

Go to file

Add file ↗

Code

...



thadson usgs and claude lint(ruff): select the checks no other gate covers, and pin the linter (75e56ab · 2 weeks ago 544 Commits

 .github/workflows	lint(ruff): select the checks no other gate co...	2 weeks ago
 .gitlab	Domain and code review	3 years ago
 dataretrieval	lint(ruff): select the checks no other gate co...	2 weeks ago
 demos	docs: copy edits across docstrings, comme...	2 weeks ago
 docs	lint(ruff): select the checks no other gate co...	2 weeks ago
 tests	lint(ruff): select the checks no other gate co...	2 weeks ago
 .gitignore	feat(config)!: resolve settings through a lay...	last month
 .importlinter	fix: keep leading zeros in every code colum...	2 weeks ago
 .pre-commit-config.yaml	lint(ruff): select the checks no other gate co...	2 weeks ago

 .pyscn-known-clones.json	ci(health): baseline the accepted clone grou...	last month
 AGENTS.md	lint(ruff): select the checks no other gate co...	2 weeks ago
 CONTEXT.md	docs: copy edits across docstrings, comme...	2 weeks ago
 CONTRIBUTING.md	docs: copy edits across docstrings, comme...	2 weeks ago
 DISCLAIMER.md	Update DISCLAIMER.md	3 years ago
 LICENSE.md	HTML Rendered Documentation via Sphinx...	4 years ago
 NEWS.md	lint(ruff): select the checks no other gate co...	2 weeks ago
 README.md	docs: copy edits across the ADRs, guides, a...	3 weeks ago
 code.json	Update code.json to include both dev and ...	2 years ago
 pyproject.toml	lint(ruff): select the checks no other gate co...	2 weeks ago
 setup.py	lint(ruff): select the checks no other gate co...	2 weeks ago

READMEContributingLicense

dataretrieval: Download hydrologic data

pypi

v1.3.0

condaconda-forgenv1.3.0

downloads526k

What is dataretrieval?

dataretrieval simplifies loading hydrologic data into Python. Like the original R version [dataRetrieval](#) , it retrieves the major U.S. Geological Survey (USGS) hydrology data types available on the Web. It also retrieves data from the Water Quality Portal (WQP), the National Ground-Water Monitoring Network (NGWMN), and the Network Linked Data Index (NLDI).

Check the [NEWS](#) for all updates and announcements.

Installation

Install dataretrieval using pip:

```
pip install dataretrieval
```

Or conda:

```
conda install -c conda-forge dataretrieval
```

Or directly from GitHub:

```
pip install git+https://github.com/DOI-USGS/dataretrieval-python.git
```

Usage Examples

Water Data API

Access USGS water-monitoring data.

Important: Users are strongly encouraged to obtain an API key for higher rate limits. [Register for an API key](#), then supply it in whichever of these ways suits you. They are listed from highest to lowest precedence, so an explicit block or deployment environment can override a file without editing it:

```
# 1. a configure() block - for one call, an interactive prompt, or when
#    different threads/tasks need different credentials.
from getpass import getpass

import dataretrieval
from dataretrieval import Configuration, waterdata

with dataretrieval.configure(Configuration(api_key=getpass("USGS API key: ")))
    df, metadata = waterdata.get_daily(monitoring_location_id="USGS-01646500")
```

https://github.com/DOI-USGS/dataretrieval-python

2/6

```
# 2. an environment variable (the R dataRetrieval package uses the same
#     variable, so one export serves both)
export API_USGS_PAT="your_api_key_here"
```

```
# 3. ~/.dataretrieval/config.toml - keeps the key out of your shell
#     environment, where every process you start inherits it.
#     Restrict it afterwards:  chmod 600 ~/.dataretrieval/config.toml
api_key = "your_api_key_here"
```

`dataretrieval.show_configuration()` reports what is in effect and where each setting came from, without printing the key. Concurrency, retries, and the progress line are configured the same way, and can be narrowed to one service: a `configure()` block takes at most one configuration per adapter, each either built in code or loaded by name from a profile in the file.

```
from dataretrieval.ngwmn import NgwmnConfiguration
from dataretrieval.waterdata import WaterdataConfiguration

with dataretrieval.configure(
    WaterdataConfiguration.load("overnight"), # a profile in config.toml
    NgwmnConfiguration(concurrency=2), # built here
):
    ...
```

See the [configuration guide](#).

The following example retrieves daily streamflow data for a specific monitoring location. The `/` in the `time` argument separates the start and end of the desired range:

```
from dataretrieval import waterdata

# Get daily streamflow data (returns DataFrame and metadata)
df, metadata = waterdata.get_daily(
    monitoring_location_id="USGS-01646500",
    parameter_code="00060", # Discharge
    time="2024-10-01/2025-09-30",
)

print(f"Retrieved {len(df)} records")
print(f"Site: {df['monitoring_location_id'].iloc[0]}")
print(f"Mean discharge: {df['value'].mean():.2f} {df['unit_of_measure'].iloc[0]}")
```

Retrieve streamflow at multiple locations from October 1, 2024 to the present:

```
df, metadata = waterdata.get_daily(
    monitoring_location_id=["USGS-13018750", "USGS-13013650"],
    parameter_code="00060",
    time="2024-10-01/..",
)

print(f"Retrieved {len(df)} records")
```

Retrieve location information for all monitoring locations categorized as stream sites in Maryland:

```
# Get monitoring location information
df, metadata = waterdata.get_monitoring_locations(
    state="Maryland", # full name, postal code ('MD'), or FIPS ('24')
    site_type_code="ST", # Stream sites
)

print(f"Found {len(df)} stream monitoring locations in Maryland")
```

Finally, retrieve continuous (a.k.a. "instantaneous") data for one location. We *strongly advise* breaking continuous data requests into smaller time windows to avoid timeouts and other issues:

```
# Get continuous data for a single monitoring location and water year
df, metadata = waterdata.get_continuous(
    monitoring_location_id="USGS-01646500",
    parameter_code="00065", # Gage height
    time="2024-10-01/2025-09-30",
)

print(f"Retrieved {len(df)} continuous gage height measurements")
```

Speeding up large downloads with `parallel_chunks`

By default the getters split a multi-value request only as far as the server's ~8 KB URL limit forces — the fewest sub-requests. For a large, paginated pull, that default is needlessly conservative: every sub-request pages through its own results, so dividing the query into more, smaller sub-requests lets those pages be fetched in parallel. `parallel_chunks(n)` opts a single call into that finer split, fanning it out into `n` sub-requests. The finer split helps only when the result is large enough to span many pages *and* the query has a multi-value argument to divide, such as a list of monitoring locations. On a small query — or one with nothing to split — it only adds requests, so `parallel_chunks` is a deliberate, scoped `with` block, never the default.

```
from dataretrieval import waterdata

# All stream gages in Ohio, then 20 years of their daily discharge – large
# enough to span many pages, so it benefits from a finer split.
sites, _ = waterdata.get_monitoring_locations(state="Ohio", site_type_code="ST")

with waterdata.parallel_chunks(32): # request up to 32 optional chunks
    df, md = waterdata.get_daily(
        monitoring_location_id=sites["monitoring_location_id"],
        parameter_code="00060", # discharge
        time="2004-01-01/2023-12-31",
    )
```

`n` is the number of sub-requests to fan the call out into, capped by how many values there are to split. Each sub-request costs a request against your hourly [rate limit](#). How many run *at once* is capped separately by `API_USGS_CONCURRENT` (default 32), so the useful range is roughly 2 up to that value.

Benchmark — a fixed 271-site subset of Ohio stream gages (`get_daily`, `parameter_code="00060"`), with a small fixed page size (`limit=250`) so every run fetches roughly the same number of pages (isolating the effect of parallelism). Each `n` ran against its own cold 1-year time window, so no run is served from the server's data-window cache:

n	optional fan-out	pages	wall-clock	speedup
off	1	~30	9.5 s / 9.1 s (2 runs)	1×
8	8	~32	2.2 s / 1.9 s	~4.5×
32	32	54	1.2 s	~8×

The gain comes from overlapping each sub-request's per-page latency and server-side work. The exact multiplier therefore scales with how many pages the pull spans: a larger pull (more pages) has more parallelism to exploit. The extra sub-requests each cost quota, so reserve a large `n` for pulls you know are large.

Visit the [API Reference](#) for more information and examples on available services and input parameters.

For verbose troubleshooting and support — including the request URL sent to the API — enable debug-level [logging](#):

```
import logging

logging.basicConfig(level=logging.DEBUG)
```

National Ground-Water Monitoring Network (NGWMN)

Access groundwater data aggregated from many state, federal, and local agencies. NGWMN uses the same OGC engine as the Water Data API, so chunking and pagination behave the same way:

```
from dataretrieval import ngwmn

# Find the groundwater monitoring sites in a state
# (state accepts a full name, a postal code like 'WI', or a FIPS code like '55')
sites, metadata = ngwmn.get_sites(state="Wisconsin")

print(f"Found {len(sites)} NGWMN sites in Wisconsin")

# Pull water levels from the first twenty sites over a time window.
water_levels, metadata = ngwmn.get_water_level(
    monitoring_location_id=sites["monitoring_location_id"][:20],
    datetime=["2022-01-01", "2024-01-01"],
)

print(f"Retrieved {len(water_levels)} water-level observations")
```

Water Quality Portal (WQP)

Access water quality data from multiple agencies:

```
from dataretrieval import wqp

# Find water quality monitoring sites (returns a DataFrame and metadata)
sites, metadata = wqp.what_sites(
    statecode="US:55", # Wisconsin
    siteType="Stream",
```

```

)

print(f"Found {len(sites)} stream monitoring sites in Wisconsin")

# Get water quality results
results, metadata = wqp.get_results(
    siteid="USGS-05427718", characteristicName="Temperature, water"
)

print(f"Retrieved {len(results)} temperature measurements")

```

Network Linked Data Index (NLDI)

Discover and navigate hydrologic networks:

```

from dataretrieval import nldi

# Get watershed basin for a stream reach
basin = nldi.get_basin(
    feature_source="comid",
    feature_id="13293474", # NHD reach identifier
)

print(f"Basin contains {len(basin)} feature(s)")

# Find upstream flowlines
flowlines = nldi.get_flowlines(
    feature_source="comid",
    feature_id="13293474",
    navigation_mode="UT", # Upstream tributaries
    distance=50, # km
)

print(f"Found {len(flowlines)} upstream tributaries within 50km")

```

Water Use (NWDC)

Retrieve modeled water-use estimates from the National Water Availability Assessment Data Companion:

```

from dataretrieval import nwdc

# Monthly public-supply withdrawals for Rhode Island, split into
# groundwater and surface-water sources (returns a DataFrame and metadata).
df, metadata = nwdc.get_wateruse(
    model="wu-public-supply-wd",
    variable=["pswdtot", "pswdgw", "pswdsw"],
    state="RI", # name/postal/FIPS; pass a list to fan out over several areas
    start_date="2020-01",
    time_resolution="monthly",
)

print(f"Retrieved {len(df)} records across {df['huc12_id'].nunique()} watersheds")

# Aggregate the HUC12 grid to a statewide monthly total (million gallons/day)
statewide = df.groupby("year_month")["pswdtot_mgd"].sum()
print(statewide.head())

```

Available Data Services

Modern USGS Water Data APIs — `dataretrieval.waterdata`

- `get_daily` : Daily statistical summaries (mean, min, max)
- `get_continuous` : High-frequency continuous (instantaneous) values
- `get_field_measurements` : Discrete measurements from field visits
- `get_peaks` : Annual peak streamflow
- `get_monitoring_locations` : Site information and metadata
- `get_time_series_metadata` : A location's available data parameters
- `get_latest_daily` : Most recent daily statistical summary
- `get_latest_continuous` : Most recent high-frequency value
- `get_stats_por` / `get_stats_date_range` : Daily, monthly, and annual statistics
- `get_samples` : Discrete USGS water-quality samples
- `get_ratings` : Stage-discharge rating curves

National Ground-Water Monitoring Network (NGWMN) — `dataretrieval.ngwmn`

- `get_sites` : Groundwater monitoring-location metadata across many agencies

- `get_water_level` : Depth-to-water and water-level observations
- `get_lithology` : Geologic-material logs by depth interval
- `get_well_construction` : Casing, screen, and build-out records
- `get_providers` : Contributing data-provider organizations

Legacy NWIS Services (Deprecated) — `dataretrieval.nwis`

- `get_dv` : Legacy daily statistical data
- `get_iv` : Legacy continuous (instantaneous) data
- `get_info` : Basic site information
- `get_stats` : Statistical summaries
- `get_discharge_peaks` : Annual peak discharge events

Water Quality Portal — `dataretrieval.wqp`

- `get_results` : Water-quality analytical results from USGS, EPA, and other agencies
- `what_sites` : Monitoring-location information
- `what_organizations` : Data-provider information
- `what_projects` : Sampling-project details

Network Linked Data Index (NLDI) — `dataretrieval.nldi`

- `get_basin` : Watershed boundary for a point or feature
- `get_flowlines` : Upstream/downstream flowline navigation
- `get_features` : Find monitoring sites, dams, and other features along the network
- `get_features_by_data_source` : Features from a specific data source

NWDC (National Water Availability Assessment Data Companion) — `dataretrieval.nwdc`

- `get_wateruse` : Modeled water-use estimates — public-supply, irrigation, and thermoelectric withdrawals and consumptive use — on a national 12-digit hydrologic-unit (HUC12) grid, summarizable to counties, states, or coarser hydrologic units

More Examples

Explore additional examples in the [demos](#) directory, including Jupyter notebooks demonstrating advanced usage patterns.

Getting Help

- Issue tracker: Report bugs and request features at <https://github.com/DOI-USGS/dataretrieval-python/issues>
- Documentation: <https://doi-usgs.github.io/dataretrieval-python/>

Contributing

Contributions are welcome! See [CONTRIBUTING.md](#) for development guidelines.

Acknowledgments

This material is partially based upon work supported by the National Science Foundation (NSF) under award 1931297. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the NSF.

Disclaimer

This software is preliminary or provisional and is subject to revision. It is being provided to meet the need for timely best science. The software has not received final approval by the U.S. Geological Survey (USGS). No warranty, expressed or implied, is made by the USGS or the U.S. Government as to the functionality of the software and related material nor shall the fact of release constitute any such warranty. The software is provided on the condition that neither the USGS nor the U.S. Government shall be held liable for any damages resulting from the authorized or unauthorized use of the software.